

# Adaptive Learning of Activation Functions via Genetic Algorithms

Caciolli Adele, Cerretani Caterina, Petrilli Niccolò

Department of Information Engineering and Mathematics, University of Siena

## ABSTRACT

This project employs a genetic algorithm to develop adaptive activation functions in neural networks, enabling them to evolve and adjust to the specific requirements of the problem. Activation functions are represented as a DNA, a discrete string of values, which is transformed into a continuous function usable during the training of the network.

The genetic algorithm operates through selection, crossover, and mutation to optimize the activation functions over time. Networks are evaluated based on their performance, measured using a fitness criterion, and the best-performing ones contribute to the next generation.

This approach enables the automatic exploration of new activation function shapes, potentially more effective than traditional ones, providing a flexible and adaptable method to enhance the performance of neural networks.

## 1. Introduction

The project aims to explore an approach in machine learning, with the goal of developing a method for the adaptive learning of activation functions within multilayer perceptron (MLP) neural networks through the use of genetic algorithms [1].

Traditionally, the activation functions used in MLPs are fixed and pre-defined (e.g., sigmoid or ReLU [2]). However, their static nature can represent a significant limitation.

The main goal of the project is to enable neural networks to autonomously learn the shape of their activation functions, thereby improving their adaptability and performance during the learning process.

The proposed methodology combines backpropagation techniques with genetic algorithms [3]. In particular, the process begins with the creation of an initial population (Generation 0), composed of a defined number of neural networks, each characterized by a randomly generated activation function. Subsequently, new populations with progressively evolved activation functions are generated through mechanisms inspired by natural selection, as tournament selection and genetic crossover [4].

The fitness of each network is evaluated based on the mean squared error (MSE) on the training data, ensuring the selection of the most promising architectures. In summary, the project demonstrates that integrating genetic algorithms with MLPs can represent a significant advancement in machine learning, paving the way for new techniques in the dynamic adaptation of neural architectures.

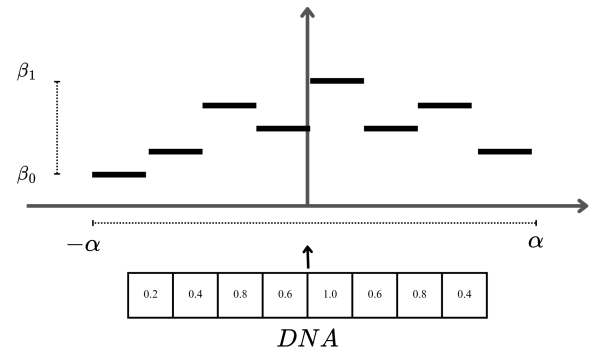
The paper is organized as follows:

- Section 2 (Methodology) explains the evolutionary process, detailing how the activation functions are generated and how the genetic algorithms are applied iteratively.
- Section 3 (Experimental Setup) describes how the experimental phase is conducted, detailing the performed experiments and the evaluation methodology.
- Section 4 (Analysis and Conclusions) analyzes the experimental phase, discusses the observations, and summarizes the main outcomes.

## 2. Methodology

The evolutionary process is structured into several phases:

- 1. Activation Function Generation:** The initial phase involves generating the activation functions for each individual. Each function is represented by a discrete domain consisting of  $n$  values within the interval  $[-\alpha, \alpha]$  and a codomain within the interval  $[\beta_0, \beta_1]$  with granularity  $\Delta_c$ . In the context of the genetic algorithm, these functions are represented as strings of values (genes) of predefined length  $n$ , constituting the individual's "DNA". Each gene in the string is randomly selected from the codomain interval [5].



**Fig. 1** Representation of activation function through the DNA

- 2. Population Initialization:** Subsequently, the initial population (Generation 0) of neural networks is generated, each equipped with a DNA string randomly generated according to the parameters set in step 1.
- 3. DNA Mapping:** The DNA is converted into a continuous activation function using cubic interpolating polynomials, which provide a good approximation of the function.
- 4. Network Training:** The resulting neural networks are trained using the backpropagation algorithm [6] for a fixed number of epochs. Performance is evaluated based on the mean squared error (MSE) on the training dataset. This error

metric serves as the individual's fitness score for the genetic selection phase.

5. **Genetic Evolution:** The next generation is produced from the current one using standard genetic operators—selection, crossover, and mutation—as described in [5]. Only the DNA (i.e., the activation function) is inherited between generations. The neural network weights are reinitialized randomly and uniformly at each generation, meaning there is no parameter inheritance beyond the activation function. Therefore, the evolutionary process focuses exclusively on optimizing the activation functions rather than the network weights.
6. **Iteration:** This evolutionary cycle is repeated across multiple generations. At each step, the population is updated based on the evolution of the individuals' "genomes" (i.e., their activation functions) until convergence criteria are met or a predefined maximum number of generations is reached.

The Figure 2 illustrates the pseudocode that describes in detail the adopted methodology.

---

Genetic algorithm for neural network population

---

**Initialization of the first generation:**

```

population ← empty list of neural networks
for  $i \leftarrow 0$  to  $pop\_size$  do
  Generate DNA for the individual.
  Generate the corresponding activation function.
  Initialize the network and append it to population.
end for

```

**Execution of generations:**

```

for  $i \leftarrow 0$  to  $N\_gens$  do
  Training:
  for each network in population do
    Train each network obtaining its score.
  end for
  new population ← empty list of neural networks

  Selection:
  Select through tournament a percentage of individuals and add them to new population.

  Mutation:
  Select through tournament a percentage of individuals, perform mutation and add them to new population.

  Crossover:
  for a percentage of individuals do
    Select through tournament the first parent.
    Select through tournament the second parent.
    for each pair do
      Perform crossover and add the offspring to new population
    end for
  end for

  population ← new population
end for

```

---

**Fig. 2** Representation of the algorithm's pseudocode

### 3. Experimental setup

In all experiments, a multilayer perceptron (MLP) with a consistent base architecture was used: a single hidden layer with a variable activation function represented by the parameters  $(n, \Delta_c, -\alpha, \alpha, \beta_0, \beta_1)$ . The only structural parameter that varied across experiments was the number of hidden units  $h$ .

The chosen task is a regression problem, where the goal is to approximate a sinusoidal function defined over the interval  $[-\alpha, \alpha]$ . A total of 800 equally spaced points were sampled within this interval to construct the dataset used for training and validation. Each neural network was trained using the standard backpropagation algorithm. The dataset was split into 70% for training and 30% for validation. The training was conducted over a maximum of  $E = 10$  epochs, with a fixed learning rate of  $l = 0.01$ . This relatively low number of training epochs was intentionally chosen to limit the influence of training procedure on network performance, allowing the genetic algorithm to play a more important role in determining the effectiveness of each activation function.

The experiments differ in terms of population size  $p$  and the number of generations  $g$ . The evolutionary methods used by the genetic algorithm remain consistent across all experiments. For selection, a tournament selection strategy was used: each individual is selected by randomly sampling  $k$  candidates from the population and choosing the best among them. The crossover is performed by randomly selecting a continuous segment of fixed length (corresponding to 10% of the genome) from the first parent's genome, which is then inserted into the corresponding position of a copy of the second parent's genome, forming the child's genome. For mutation, a Flip Bit Mutation strategy was adopted, where a fixed number of genes are randomly selected and replaced with new values sampled from a predefined range. In all experiments, 90% of the individuals in each new generation were generated through crossover, 5% through mutation, and the remaining 5% were directly carried over from the previous generation without modification. As the fitness function of genetic algorithms, the final training loss of each neural network was used, with lower values indicating better performance.

The selected experiments are the following:

- **E1 – Baseline:** reference configuration with balanced parameters, used as a comparison point for the other experiments.
- **E2 – Extended codomain [0,2]:** evaluates whether a wider output range improves the expressiveness of the activation function.
- **E3 – Short DNA ( $n = 6$ ):** analyzes the impact of a less detailed (low-resolution) activation function.
- **E4 – Reduced population ( $p = 300$ ):** investigates the effect of a smaller population on the effectiveness of genetic evolution.
- **E5 – Larger network ( $h = 20$ ):** tests whether a network with greater capacity benefits more from evolved activations.
- **E6 – Symmetric codomain  $[-1, 1]$ :** tests the effect of allowing negative outputs in order to potentially increase the flexibility of activation function.

The specific parameters used for each experiment are summarized in Table 1.

**Table 1** Experiments parameters (E1–E6)

| Params     | E1   | E2   | E3   | E4  | E5   | E6   |
|------------|------|------|------|-----|------|------|
| $n$        | 12   | 12   | 6    | 12  | 12   | 12   |
| $\Delta_c$ | 0.1  | 0.1  | 0.1  | 0.1 | 0.1  | 0.1  |
| $-\alpha$  | -4   | -4   | -4   | -4  | -4   | -4   |
| $\alpha$   | 4    | 4    | 4    | 4   | 4    | 4    |
| $\beta_0$  | 0    | 0    | 0    | 0   | 0    | -1   |
| $\beta_1$  | 1    | 2    | 1    | 1   | 1    | 1    |
| $h$        | 12   | 12   | 12   | 12  | 20   | 12   |
| $p$        | 1000 | 1000 | 1000 | 300 | 1000 | 1000 |
| $g$        | 30   | 30   | 30   | 30  | 30   | 30   |
| $k$        | 12   | 12   | 12   | 8   | 12   | 12   |

The analysis of the experiments is carried out by monitoring the evolution of each population over successive generations. In particular, two key indicators are tracked: the average score of the population (Mean MSE on the training set) and the minimum score, corresponding to the best-performing individual. These values are stored for each generation, enabling the analysis of convergence trends and performance variability. Additionally, the shape of the evolved activation functions is visualized to qualitatively assess smoothness, expressiveness, and stability across generations.

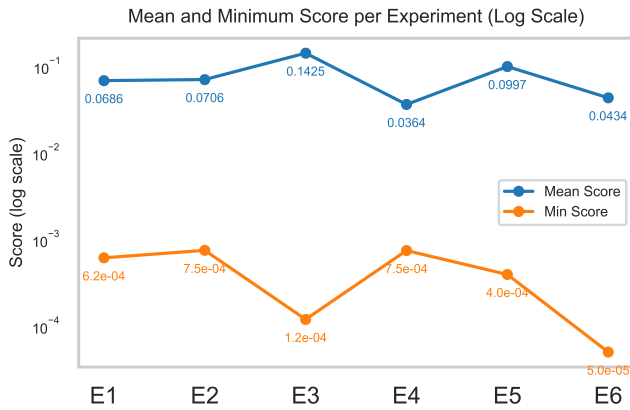
To evaluate each trained neural network, a validation phase was performed using a separate validation set. During this phase, the model parameters remained fixed. The model's predictions  $\hat{y}_i$  were computed over all input samples  $x_i$  in the validation set and compared to the true targets  $y_i$ . The performance was quantitatively assessed using the Mean Squared Error (MSE), computed as:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

where  $n$  denotes the number of validation samples.

Furthermore, a final comparison is performed between the best evolved neural network (in terms of validation error) and a reference network using a standard sigmoid activation function. This comparison includes both training and validation MSEs, as well as a visual analysis of the predicted outputs. The goal is to assess whether the evolved activation functions provide a tangible advantage over commonly used fixed activations.

#### 4. Analysis and conclusion

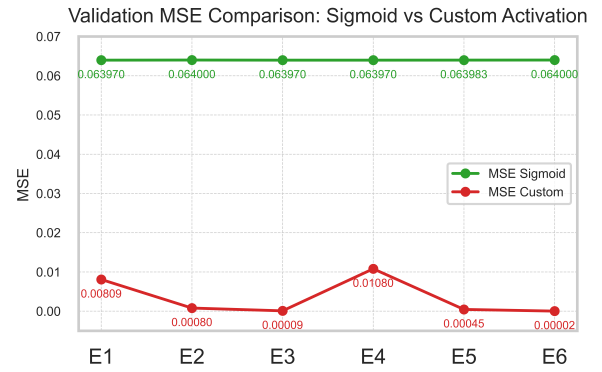


**Fig. 3** The mean score reflects the average performance of all individuals, while the minimum score indicates the best-performing individual. Lower scores indicate better solutions.

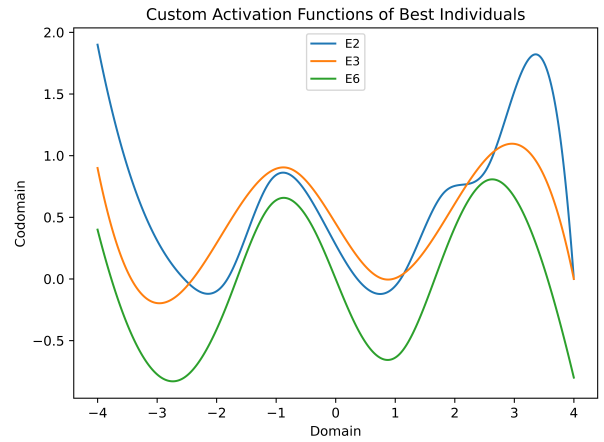
The table of results summarizes the outcomes of experiments E1 to E6, each exploring a different configuration of the activation function evolution process. Figure 3 and 4 show the numerical results. Figure 5 shows the custom activation functions of the best experiments

##### 4.1. Individual Experiment Analysis

In the experiment E1 (baseline) the average score was moderately low, and the best-performing individual achieved a minimum score of 0.00062. The validation error was significantly



**Fig. 4** Each point represents the final MSE after training. The custom activation functions consistently yield lower MSE values, indicating improved generalization.



**Fig. 5** Activation functions evolved by the best individuals from experiments E2, E3, and E6.

better than that of the sigmoid reference model, confirming the feasibility of the evolutionary approach.

In the experiment E2 (Extended Codomain) although the average score was slightly worse than E1, the best individual reached an acceptable minimum score (0.00075), and the validation performance was remarkably good, indicating better generalization despite lower training fitness. This suggests that a wider codomain might help in some cases, although not consistently across the population.

In the third experiment E3 (Short DNA) as expected, the average training score worsened significantly, showing that most individuals failed to represent effective activation shapes. Despite that, the best individual achieved an excellent minimum score (0.00012) and the lowest validation MSE of all, suggesting that even low-resolution activations can occasionally produce high-performing networks. This can highlight the importance of selection mechanisms to capture rare but promising individuals.

The fourth experiment E4 (Reduced Population) resulted in the best average training score across all experiments. However, the validation MSE was worse than E1, that can indicate overfitting or reduced generalization. The best individual was good but not the best overall. These results confirm that while smaller populations may converge faster, they can also limit diversity and robustness.

The experiment E5 aimed to understand if a larger network could better exploit the evolved activations. The average training score

was worse, but the best individual achieved a very good minimum score. The validation MSE was among the best, it can suggest the higher-capacity networks may benefit from evolved activations, but it could require a longer training to realize their full potential.

The configuration used in the experiment E6 (Symmetric Codomain) allowed activation values to be both positive and negative, similar to the behavior of functions like  $\tanh$ . It has produced the best individual of all and the best validation performance. The average training score was also among the lowest. These results suggest that symmetric activations are well suited to the project task, perhaps thanks to their ability to model negative and positive values in a balanced way.

#### 4.2. Comparative Analysis and Conclusions

In all experiments, the evolutionary genetic algorithm approach outperformed the baseline model in terms of validation. The most improvements came from E6, E3, and E2, where each produces individuals with excellent generalization capabilities. An interesting fact, experiment E3 had the worst average score, but the best generalization. This can highlight how, thanks to the genetic diversity in the population, the algorithm can still generate a few exceptional individuals, even if most are poor.

E4 demonstrated that reducing the population size can accelerate convergence but losing generalization. E5 confirmed that network capacity alone does not guarantee better performance unless paired with suitable activation shapes. E2 and E6 underline the importance of designing flexible activation ranges: broader or symmetric codomains can offer the expressiveness needed to fit complex patterns like the sine function.

In conclusion, this project explored the use of genetic algorithms to evolve custom activation functions in neural networks, with the goal of improving adaptability and performance without using on predefined shapes. The experimental results confirmed that this approach is not only feasible but often superior in terms of generalization. In several cases, evolved activation functions outperformed the sigmoid baseline, especially when the function use symmetric or extended codomains. Even with limited training (10 epochs), the evolutionary process successfully identified activation shapes capable of capturing the structure of data. The ability to discover rare but highly effective individuals, as seen in E3, highlights the strength of genetic diversity in the search process. Overall, this work shows that adaptive activation functions can improve neural network flexibility and performance, and that genetic algorithms offer a powerful tool for discovering them automatically.

#### References

- [1] D. E. Goldberg, *Genetic Algorithms in Search, Optimization and Machine Learning*, Addison-Wesley, 1989.
- [2] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.
- [3] X. Yao, "Evolving artificial neural networks," *Proceedings of the IEEE*, vol. 87, no. 9, pp. 1423–1447, 1999.
- [4] X. Glorot, A. Bordes, and Y. Bengio, "Deep sparse rectifier neural networks," in *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, pp. 315–323, 2011.
- [5] D. E. Goldberg, *Genetic Algorithms in Search, Optimization and Machine Learning*, Addison-Wesley, 1989.
- [6] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, no. 6088, pp. 533–536, 1986.